

CSIRO Image2Biomass Prediction — 5th Place Solo Gold Medal Solution

Rank	4
Team	yanqiangmiffy
Author	yanqiangmiffy
Topic ID	670668
Version	Japanese Translation

Source: <https://www.kaggle.com/competitions/csiro-biomass/discussion/670668>
Retrieved with Kaggle CLI on 2026-07-18.

Kaggle Discussionの主投稿と、技術的補足を含む選定済みQ&Aを日本語へ翻訳した版です。code、URL、model名、識別子、数値は原文を保持しています。

まず、このコンペを開催したKaggleとCSIROに感謝します。農業AIの非常に意義深い応用scenarioです。

1. はじめに

Kaggle CVコンペへの参加は初めてで、solo 5位（Private LB: 0.76）を獲得できたことを嬉しく、幸運に思います。期間中に100を超えるtraining experiment（exp-v1~exp-v188）を行いました。このコンペからcomputer visionについて多くを学びました。以下で詳細を共有します。

Submissions evaluated for final score

All		Successful	Selected	Errors	Private Score ▾		
Submission and Description		Private Score ⓘ		Public Score ⓘ	Selected		
✓	exp177-csiro-inference-cv - Version 1 Succeeded · 7d ago · v164+調整損失权重, 加大损失权重 total	0.66		0.75	☐		
✓	CSIRO-Ensemble-Models-V11 - Version 26 Succeeded · 2d ago · 3tta+v177+148v2	0.66		0.76	☐		
✓	exp185-csiro-inference-cv - Version 1 Succeeded · 2d ago · exp-v185 0.9674 0.xx v181+ng segmentation_loss_weight 0.3	0.66		0.74	☐		
✓	CSIRO-Ensemble-Models-V9 - Version 8 Succeeded · 2d ago · 相比v6替换为exp148v2	0.66		0.76	☐		
✓	CSIRO-Ensemble-Models-V12 - Version 7 Succeeded · 1d ago · 根据训练集设置阈值	0.66		0.76	☒		
✓	CSIRO-Ensemble-Models-V15 - Version 1 Succeeded · 12h ago · 142 two tta,在CSIRO-Ensemble-Models-V12 - Version 7基础上 (0.76)	0.66		0.76	☐		
✓	exp181-csiro-inference-cv - Version 2 Succeeded · 4d ago · v174+unet dinov3 unet 1024 ng	0.66		0.74	☐		
✓	CSIRO-Ensemble-Models-V9 - Version 6 Succeeded · 3d ago · two tta调整权重	0.66		0.76	☐		
✓	CSIRO-Ensemble-Models-V9 - Version 3 Succeeded · 6d ago · two tta	0.65		0.76	☐		
✓	exp146-csiro-inference-cv - Version 3 Succeeded · 14d ago · dead/total ratio预测	0.65		0.74	☐		
✓	CSIRO-Ensemble-Models-V10 - Version 4 Succeeded · 5d ago	0.65		0.76	☐		
✓	exp164-csiro-inference-cv - Version 2 Succeeded · 9d ago · exp-v164 0.9764 0.71 0.75 v147+使用多种数据增强方式进行训练	0.65		0.75	☐		
✓	CSIRO-Ensemble-Models-V16 - Version 9 Succeeded · 9h ago · lgb online	0.65		0.76	☒		

2. コンペの理解

2.1 Problem description

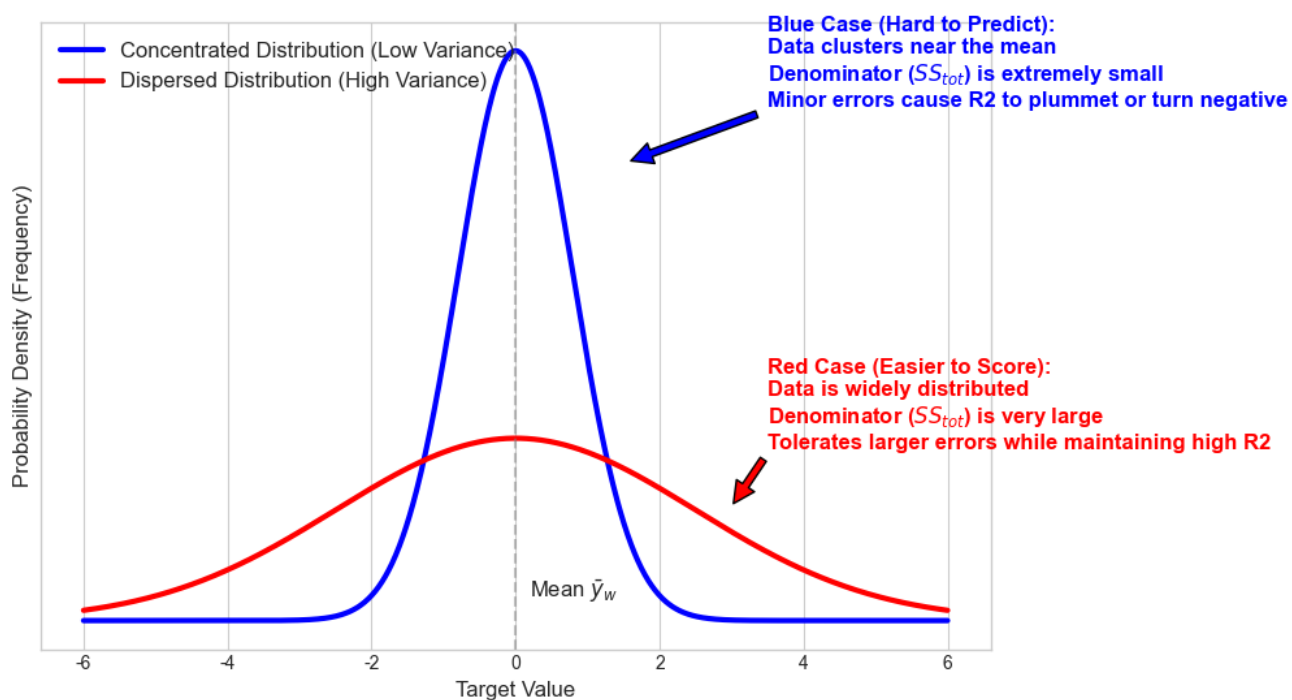
pasture imageから次の5つの主要biomass componentを予測します。

- **Dry_Green_g** — dry green vegetation (cloverを除く)
- **Dry_Dead_g** — dry dead material
- **Dry_Clover_g** — dry clover biomass
- **GDM_g** — green dry matter
- **Dry_Total_g** — total dry biomass

2.2 Dataの特徴

- **小さなtraining set:** 約800+画像
- **Image size:** 元画像は2000×1000 pixel
- **Distribution shift:** train/test分布に大きな差
- **Multi-state data:** NSW、Tas、Vic、WAのdata
- **Target間の物理constraint:**
 - `Dead = Total - GDM`
 - `Clover = GDM - Green`

Trend of R^2 Metric Relative to Data Concentration



2.3 Evaluation metric

5targetの R^2 をweighted averageするweighted R^2 scoreです。

3. Core improvement technique

広範なablation experimentの結果、次が安定してscoreを改善しました。

3.1 Image preprocessing — Timestamp removal

元画像にはnoiseとなるorange date timestampがあります。HSV color detectionとimage inpaintingで除去しました。

```
def clean_image(img):  
    """  
    Image preprocessing: Remove bottom artifacts and date stamp.  
    """  
    # Detect orange date stamp in HSV space  
    hsv = cv2.cvtColor(img, cv2.COLOR_RGB2HSV)  
    # Define orange range  
    lower = np.array([5, 150, 150])  
    upper = np.array([25, 255, 255])  
    mask = cv2.inRange(hsv, lower, upper)  
  
    # Dilate mask to cover text edges  
    mask = cv2.dilate(mask, np.ones((3, 3), np.uint8), iterations=2)  
  
    # Inpaint if timestamp detected  
    if np.sum(mask) > 0:  
        img = cv2.inpaint(img, mask, 3, cv2.INPAINT_TELEA)  
  
    return img
```



+ Code + Markdown

[3]:

3.2 複数data augmentation

小さなtraining setでoverfittingを防ぐにはdata augmentationが不可欠です。TTA-style training strategyを使いました。

```
def get_tta_transforms(img_size: int) -> list[A.Compose]:  
    """Returns a list of transform pipelines for TTA-style training."""  
    normalize = A.Normalize(  
        mean=[0.485, 0.456, 0.406],  
        std=[0.229, 0.224, 0.225]  
    )
```

```

# View 0: Original
original_view = A.Compose([
    A.Resize(img_size, img_size),
    normalize,
    ToTensorV2()
])

# View 1: Horizontal Flip
hflip_view = A.Compose([
    A.Resize(img_size, img_size),
    A.HorizontalFlip(p=1.0),
    normalize,
    ToTensorV2()
])

# View 2: Vertical Flip
vflip_view = A.Compose([
    A.Resize(img_size, img_size),
    A.VerticalFlip(p=1.0),
    normalize,
    ToTensorV2()
])

# View 3: Rotate 90 degrees
rotate90_view = A.Compose([
    A.Resize(img_size, img_size),
    A.Rotate(limit=(90, 90), p=1.0, border_mode=0),
    normalize,
    ToTensorV2()
])

return [hflip_view, vflip_view, rotate90_view, original_view]

```

Training strategy:

- 最初の15 epochsはaugmented view (hflip、vflip、rotate90) で学習し、dataを実質4倍化。
- その後のepochはoriginal viewだけを使用。
- 重要な発見:** augmentation methodをrandomに選ぶ方が、epoch順に切り替えるより良かった (exp-v174) 。

3.3 Multi-view input architecture

stereo visionを模した左右分割に加え、full imageを3つ目のinput streamとして加えました。

```

class BiomassDataset(Dataset):
    """Three-stream dataset: splits 2000x1000 image into left, right, and full."""

    def __getitem__(self, idx: int):
        image = self._load_image(self.image_paths[idx])
        image = clean_image(image)

        height, width = image.shape[:2]
        mid = width // 2
        left = image[:, :mid]      # Left half
        right = image[:, mid:]     # Right half
        full_image = image.copy()  # Full image

        # Apply same transform to all three views

```

```

left = self.transform(image=left)["image"]
right = self.transform(image=right)["image"]
full_image = self.transform(image=full_image)["image"]

return left, right, full_image, train_tgt, eval_tgt, species_tgt

```

局所detail (left/right) とglobal context (full view) を同時に学べます。

3.4 Mixture-of-Experts (MoE) architecture

設計意図: 入力画像の特徴 (green grassが多い、dead grassが多いなど) に応じて最適なexpertへ自動的に切り替え、multi-task learningのfeature conflictを解決します。

```

class MetaMoE(nn.Module):
    """Mixture-of-Experts head for main targets, outputs mean and variance."""

    def __init__(self, in_dim: int, hidden: int, num_experts: int,
                  out_dim: int = 1, dropout: float = 0.2):
        super().__init__()
        self.num_experts = num_experts

        # Gating network
        self.gate = nn.Sequential(
            nn.LayerNorm(in_dim),
            nn.Linear(in_dim, 256),
            nn.ReLU(),
            nn.Linear(256, num_experts),
        )

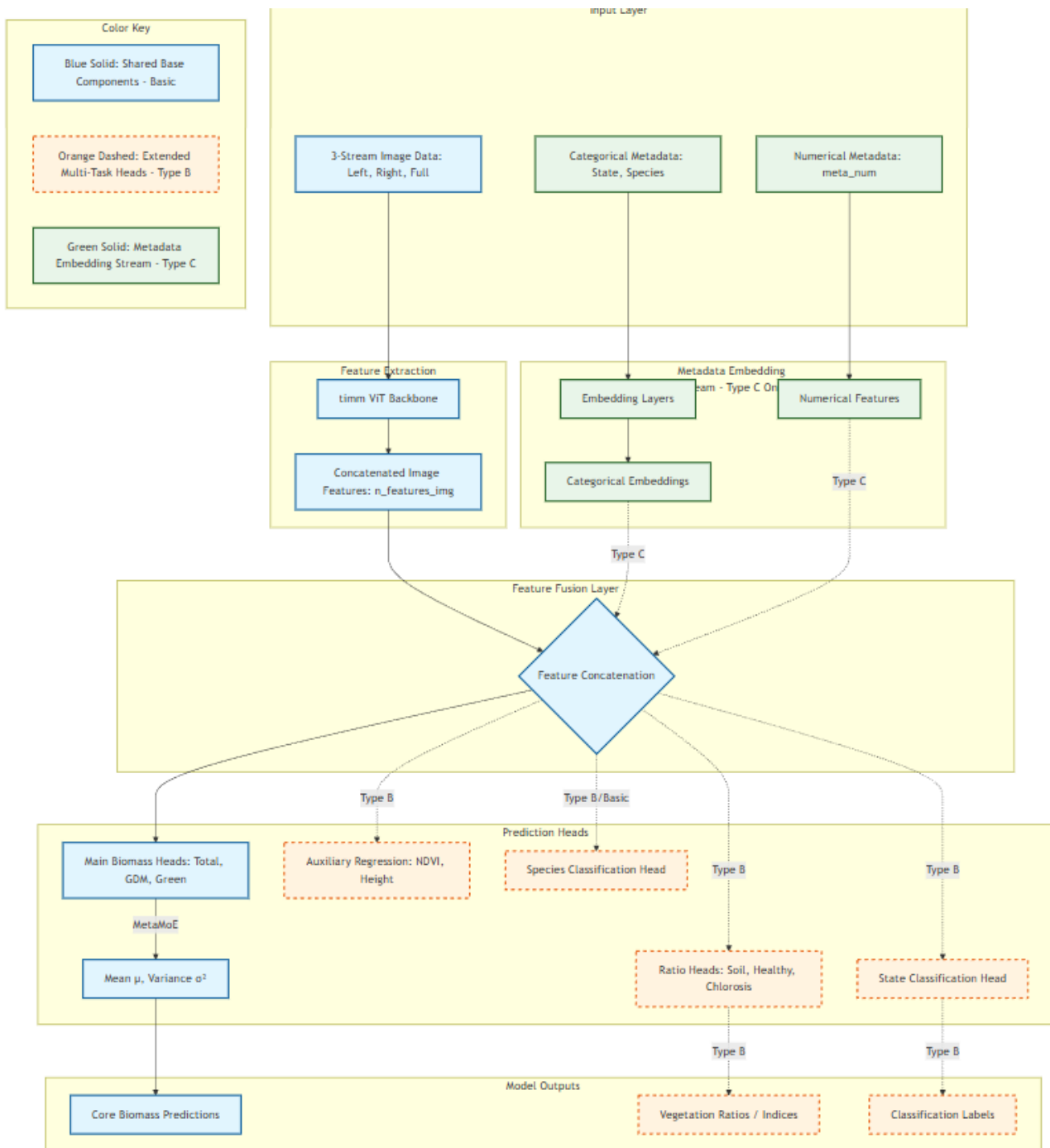
        # Expert networks (each outputs mean and variance)
        self.experts = nn.ModuleList([
            nn.Sequential(
                nn.LayerNorm(in_dim),
                nn.Linear(in_dim, hidden),
                nn.ReLU(),
                nn.Dropout(dropout),
                nn.Linear(hidden, 256),
                nn.ReLU(),
                nn.Linear(256, 2), # [mean, var_logit]
            )
            for _ in range(num_experts)
        ])

    def forward(self, x: torch.Tensor):
        gate_logits = self.gate(x)
        gate_weights = F.softmax(gate_logits, dim=1).unsqueeze(-1)
        expert_outs = torch.stack([expert(x) for expert in self.experts], dim=1)
        out = (expert_outs * gate_weights).sum(dim=1)

        pred_mean = out[:, 0:1]
        pred_var_logit = out[:, 1:2]
        pred_var = F.softplus(pred_var_logit) + 1e-6 # Ensure variance is positive

        return pred_mean, pred_var

```



3 model variant:

- **Type A (BiomassModelBasic)** : 基本3-stream model (exp-v147、exp-v164、exp-v174)
- **Type B (BiomassModelWithStateSpecies)** : State/Species classification head付き (exp-v142)
- **Type C (BiomassModelWithMetaEmbedding)** : meta feature embedding付き (exp-v148)

3.5 Auxiliary training head

追加training objectiveで豊かなfeature representationを学ばせました。

```
class BiomassModel(nn.Module):
    def __init__(self, model_name: str, pretrained: bool = True,
```

```

        pretrained_path: str = '', num_species: int = 0):
    super().__init__()
    # ... backbone initialization ...

    # Main regression heads (using MoE)
    self.head_total = MetaMoE(...)
    self.head_gdm = MetaMoE(...)
    self.head_green = MetaMoE(...)

    # Auxiliary regression heads (NDVI and Height)
    self.head_ndvi = _AuxHead(self.n_features, self.n_features // 2)
    self.head_height = _AuxHead(self.n_features, self.n_features // 2)

    # Optional classification heads
    if num_species > 0:
        self.head_species = self._make_classification_head(num_species)

```

Auxiliary target:

- Pre_GSHH_NDVI
- Height_Ave_cm
- Species classification (15 class)
- State classification (4 class)

3.6 Physics consistency loss

target間の物理関係に基づき、main lossにGaussianNLLLossを使いました。

主な利点:

- R^2 metricの**outlier error**感度に対処
- GaussianNLLにより high-noise sample を自動downweightし dirty data のbias を回避
- 物理constraintで prediction の妥当性を保ち、大誤差につながる論理矛盾を回避

```

class PhysicsConsistencyLoss(nn.Module):
    """
    Physics consistency loss using GaussianNLLLoss

    Key advantages:
    - Automatic outlier handling: model learns to predict high variance for noisy samples
    - Handles heteroscedasticity: different uncertainty for different biomass levels
    - Physical constraints: Dead = Total - GDM, Clover = GDM - Green
    """

    def __init__(self, weights: dict[str, float] = None):
        super().__init__()
        self.w = {
            "total": 0.45,
            "gdm": 0.2,
            "green": 0.1,
            "dead": 0.1,
            "clover": 0.1,
            "ndvi": 0.0,
            "height": 0.0,
            "species": 0.05
        }

```

```

self.criterion = nn.GaussianNLLLoss()

def forward(self, preds, targets, species_targets=None):
    # Direct prediction losses
    loss_total = self.criterion(pred_total_mean, gt_total, pred_total_var)
    loss_gdm = self.criterion(pred_gdm_mean, gt_gdm, pred_gdm_var)
    loss_green = self.criterion(pred_green_mean, gt_green, pred_green_var)

    # Physics constraint losses
    # Dead = Total - GDM
    pred_dead_mean = pred_total_mean - pred_gdm_mean
    pred_dead_var = pred_total_var + pred_gdm_var
    gt_dead = gt_total - gt_gdm
    loss_dead = self.criterion(pred_dead_mean, gt_dead, pred_dead_var)

    # Clover = GDM - Green
    pred_clover_mean = pred_gdm_mean - pred_green_mean
    pred_clover_var = pred_gdm_var + pred_green_var
    gt_clover = gt_gdm - gt_green
    loss_clover = self.criterion(pred_clover_mean, gt_clover, pred_clover_var)

    # Weighted total loss
    total_loss = (
        self.w["total"] * loss_total +
        self.w["gdm"] * loss_gdm +
        self.w["green"] * loss_green +
        self.w["dead"] * loss_dead +
        self.w["clover"] * loss_clover
    )
    return total_loss

```

3.7 Layer-wise learning rate

pretrained backboneと新規headへ異なるlearning rateを設定しました。

```

param_groups = []
for name, param in model.named_parameters():
    if 'backbone' in name:
        # Backbone uses smaller lr to preserve pretrained capability
        param_groups.append({
            'params': param,
            'lr': cfg.finetune_lr * cfg.backbone_lr_mult # typically 0.1
        })
    elif 'head' in name:
        # Head uses larger lr for fast fitting
        param_groups.append({
            'params': param,
            'lr': cfg.finetune_lr * cfg.head_lr_mult # typically 1.0
        })
    else:
        param_groups.append({'params': param, 'lr': cfg.finetune_lr})

optimizer = optim.AdamW(param_groups, weight_decay=cfg.weight_decay)

```

backboneは小さいlrでgeneral feature extraction能力を保ち、新規headは大きいlrでtaskへ素早くfitさせます。

3.8 Post-processing strategy

3.8.1 State-based post-processing

WAは他Stateと異なるため、Dead=0、Total=GDMとしました。

```
# WA state special handling: Dead is 0, Total equals GDM
if predicted_states_df is not None:
    final_pred = final_pred.merge(predicted_states_df, on='image_path', how='left')
    is_wa = final_pred['Predicted_State'] == 'WA'
    final_pred.loc[is_wa, 'Dry_Death_g'] = 0
    final_pred.loc[is_wa, 'Dry_Total_g'] = final_pred.loc[is_wa, 'GDM_g']
```

3.8.2 Statistics-based range clipping

State別training target統計の範囲へclipしました。

```
state_stats = {
    'Tas': {'Dry_Clover_g': {'min': 0.0, 'max': 71.79}, ...},
    'NSW': {'Dry_Clover_g': {'min': 0.0, 'max': 10.10}, ...},
    'WA': {'Dry_Clover_g': {'min': 0.0, 'max': 58.88}, ...},
    'Vic': {'Dry_Clover_g': {'min': 0.0, 'max': 67.90}, ...}
}

for state in state_stats.keys():
    state_mask = final_pred['Predicted_State'] == state
    for var in target_vars:
        min_val = state_stats[state][var]['min']
        max_val = state_stats[state][var]['max']
        final_pred.loc[state_mask, var] = final_pred.loc[state_mask, var].clip(
            lower=min_val, upper=max_val
        )
```

3.8.3 Scaling post-processing

```
def apply_post_processing(df: pd.DataFrame) -> pd.DataFrame:
    """Apply Clover and Dead post-processing rules."""
    df_out = df.copy()
    if 'Dry_Clover_g' in df_out.columns:
        df_out['Dry_Clover_g'] = df_out['Dry_Clover_g'] * 0.8
    if 'Dry_Death_g' in df_out.columns:
        mask_high = df_out['Dry_Death_g'] > 20
        df_out.loc[mask_high, 'Dry_Death_g'] *= 1.1
        mask_low = df_out['Dry_Death_g'] < 10
        df_out.loc[mask_low, 'Dry_Death_g'] *= 0.9
    return df_out
```

3.9 Full dataset training

training setが小さいため、すべてのdataを使いtraining setへoverfitさせる意図でした。

```
# train_df = df[df["fold"] != fold].reset_index(drop=True)
# valid_df = df[df["fold"] == fold].reset_index(drop=True)
```

```
train_df = df
valid_df = df
```

4. Model ensemble

4.1 最終ensemble

最終提出は6 modelのweighted ensembleです。

Model	Weight	特徴
exp-v142	0.0 (State予測のみ)	BiomassModelWithStateSpecies、State情報を抽出
exp-v148	0.15	BiomassModelWithMetaEmbedding、meta feature使用
exp-v147	0.20	BiomassModelBasic、seed=2026
exp-v174	0.25	BiomassModelBasic、random augmentation
exp-v164	0.20	BiomassModelBasic、TTA-style augmentation
exp-v177	0.20	BiomassModelBasic、調整したloss weight

4.2 TTA inference

各modelで次の3 TTAを使い、平均しました。

1. Original view
2. Horizontal flip
3. Vertical flip

4.3 Dual-GPU parallel inference

2 GPUで並列推論しました。

```
def parallel_dual_gpu_inference(dataset, model_class, model_kwargs, model_states, ...):
    """Run inference in parallel on two GPUs."""
    # Even indices -> GPU 0, Odd indices -> GPU 1
    indices_0 = list(range(0, n, 2))
    indices_1 = list(range(1, n, 2))

    with ThreadPoolExecutor(max_workers=2) as executor:
        future_0 = executor.submit(inference_single_gpu, model_0, ..., device_0)
        future_1 = executor.submit(inference_single_gpu, model_1, ..., device_1)
        results_0 = future_0.result()
        results_1 = future_1.result()

    # Merge results
    ...
```

5. 主要hyperparameter

Hyperparameter	値
Backbone	vit_large_patch16_dinov3_qkvb
Image size	1024×1024
Batch size	4
Learning rate	2e-4 (warm-up後5e-5へfine-tune)
Backbone LR multiplier	0.1
Training epochs	25
MoE experts	4
MoE hidden dim	512
Optimizer	AdamW
Weight decay	1e-4

6. 失敗した試み

6.1 効果がなかった方法

Experiment	内容	結果
exp-v36	Ratio prediction	CV 0.58→0.43
exp-v97	分割なしfull image	CV 0.57、LB 0.52 (大幅低下)
exp-v99	LR 2e-4+20 epochs	CV 0.21のみ
exp-v123	Segmentation+attention	CV 0.53、LB 0.53
exp-v168/169	Mamba Network	CV 0.80-0.89 (不安定)
exp-v180	UNet original size 1000×2000	LB 0.64 (大幅低下)

6.2 不安定だった方法

Experiment	内容	問題
Huge model (exp-v170)	vit_huge	CV 0.95だがLB 0.72
30 epochs (exp-v161)	epoch増加	overfitting、Private LB低下
各種segmentation mask	ng/xg mask	一部で有効だが不安定

6.3 学んだこと

1. **大modelが常に良いとは限らない:** small datasetではHuge modelがoverfitしやすい。
2. **Training epochを制御する:** 約25 epochsが最適balance。
3. **複雑なattention mechanismの利益は限定的:** small datasetではoverfitしやすい。
4. **Mambaなどsequence model architectureはこのproblemに不向き。**

7. 実験の進展

7.1 主要milestone

Stage	Experiment	Public LB	Private LB	主な改善
Baseline	v1	0.57	—	448 size baseline
Model upgrade	v24	0.65	—	vit_large_dinov3
Resolution	v32	0.69	—	512 size
Physics loss	v42	0.69+	—	PhysicsConsistencyLoss
MoE	v44	0.69	—	Mixture-of-Experts
1024 resolution	v108	0.73	—	1024 size+LR optimization
Full training	v136	0.74+	—	full dataset+Species
Seed optimization	v147	0.75+	—	seed=2026
Augmentation	v164	0.75	—	TTA-style augmentation
Random aug	v174	0.76	0.65	random augmentation selection
Ensemble	Final	0.76	0.66	6-model weighted ensemble

7.2 最終Private LB

最終選択した提出はPublic 0.76、Private 0.66（5位）でした。

8. まとめと経験

8.1 成功要因

1. **広範な実験:** 188を超える実験でsolution spaceを探索。
2. **物理constraint:** targetの物理関係を使うloss function。
3. **多様なensemble:** seed、architecture、training strategyの異なるmodelを融合。
4. **Data augmentation:** small datasetでは適切なaugmentationが重要。
5. **Post-processing:** data analysisに基づくpost-processing。

8.2 Submission選択

次の理由でsubmission選択は非常に難しいものでした。

- datasetが小さく、modelがoverfitしやすい。
- train/testにdistribution shiftがある。
- Public/Private LBの相関が低い。

最終的にPublic LB最高の提出を選びました。Privateは最適ではありませんでしたが（0.66、一部の失敗提出は0.65）、比較的安定していました。

8.3 Luck factor

この結果にluckがあったことは認めます。

- 多くのtop playerが最終順位でshake-upした。
- submission selectionが偶然pitfallを回避した。
- multi-model ensembleが安定性を高めた。

8.4 謝辞

Kaggle communityで惜しみなく共有してくれた皆さん、Discussionとnotebook、そして意義深いコンペを提供したCSIRO organizerに感謝します。

このsolo gold medalはGrandmasterを目指すjourneyで非常に重要です。このSolution共有が皆さんの役に立つことを願います。

Code repository: [2025-Kaggle-CSIRO-5th-solution](#)

Final submission notebook: [CSIRO-Ensemble-Models-V12](#)

補足Q&A

Resolutionの効果

chenwenqiang_001（質問・所感）：複数のtop solutionでresolutionが1024×1024でした。これが改善の鍵だと思います。

yanqiangmiffy（著者）：はい、大きいresolutionは有効でした。224、448、512、768、1024を試しました。

timmのpooling設定

chenwenqiang_001（質問）：DINOv3は512×512、768×768でpretrainされているのに、なぜ1024×1024が最良なのでしょうか。またtimm built-inのattention poolingを使いましたか。

yanqiangmiffy（著者）：次のように `img_size` を設定してloadし、average poolingを使いました。

```
self.backbone = timm.create_model(model_name, pretrained=pretrained, num_classes=0,
global_pool="avg")
```

MoE expert activationの解析

Komil Parmar（質問） : 各expertがどれほどactivateされたか解析しましたか。expert数を増やすことも試しましたか。

yanqiangmiffy（著者） : expert activationの特別な解析は行っていません。比較的単純な設定を使いました。